MultiFuel

Project Delivery and Deployment Summary

A multi-station petrol pump management application for station owners, managers, and duty staff. This document summarizes the implemented business workflows, reporting, notifications, and current deployment preparation.

Project	MultiFuel Petrol Pump Management System
Technology	Django 5.2, Python 3.11, SQLite for local use, PostgreSQL for deployment
Prepared	06 November 2026
Deployment target	Render blueprint prepared; production deployment not yet launched

Delivery status

The application includes the core operational modules, role-based dashboards, analytics and downloadable reports. A Render blueprint has been prepared for a fresh PostgreSQL deployment. Connecting the repository and creating paid hosting resources still requires action in the user's Render account.

Measured project footprint

Measure	Verified count
User roles	3: Owner, Manager, DSMS staff
Application modules	9 Django modules
Application data models	33 across the 9 modules
HTML templates	52
Application URL patterns	90, excluding Django admin
Automated test suite	133 tests passed
Analytics reports	4 report types
Export options	3 formats; 12 report-format combinations
Notification categories	6 workflow areas

1. Application roles and access

- **Owner:** controls stations, staff access, operational configuration, approvals, and payroll.
- **Manager:** works with assigned active stations and authorized reviews.
- **DSMS staff:** submits duty reports and operational records for the assigned station.
- Registration approval, active-account checks, server-side role restrictions, password reset, and CSRF-protected sign-out.

2. Station and fuel operations

- Station records with unique codes, owner scoping, and activate/suspend controls.
- Fuel types, tanks, nozzles, fuel prices with effective dates, and station lubricant catalogs.
- Supplier deliveries with invoice information, tank capacity checks, and atomic stock updates.
- An immutable inventory ledger for opening stock, deliveries, sales, testing, returns, and approved adjustments.
- Low-stock alerts and Manager-requested inventory adjustments requiring Owner review.

3. Duty reporting and cash reconciliation

- Station shifts and editable duty-report drafts with meter readings, testing quantities, and price snapshots.
- Submission, verification, correction requests, resubmission, approval, and immutable review history.
- Fuel and lubricant sales, with stock updates tied to report submission/approval workflows.
- Cash denominations, card and UPI tenders, credit sales, advances, OTP settlements, and report expenses.
- Station reconciliation policies, shortage/excess variance, and immutable authorized adjustments.

4. Credit, workforce, and expenses

- Credit customers, vehicle details, limits, report-linked fuel sales, repayments, statements, and overdue balances.

- Station-scoped attendance, leave review, employee advances, monthly payroll, payslips, and salary expense posting.
- Station expenses submitted by staff, Owner approvals, immutable decisions, and accounting summaries.

5. Analytics, exports, and notifications

Area	Implemented capability
Analytics	Sales/expenses, fuel stock and movement, duty-report reconciliation, workforce attendance/payroll.
Access scope	Owner reports are limited to owned stations; Manager reports are limited to assigned active stations.
Downloads	CSV, Excel (.xlsx), and PDF reports with date and station filters.
Notifications	In-app inbox and email notifications for account and business workflow events.

Email delivery needs SMTP configuration and a public SITE_URL. SMS is not part of the project scope.

6. Quality checks

- Django system checks and migration consistency checks were run successfully during development.
- Automated tests cover authorization, station isolation, workflows, exports, and notifications.
- The full automated test suite passed: 133 tests. The targeted health, report, and notification checks passed: 12 tests.

7. Render deployment preparation

The repository now includes a Render Blueprint for a Django web service and a fresh managed PostgreSQL database. It configures a generated Django secret, production DEBUG mode, static file collection, pre-deploy migrations, a database-backed health check, and environment slots for email settings. Local SQLite data is not included.

- In Render, connect the GitHub repository and apply the Blueprint after reviewing current web-service and database pricing.
- Set SMTP credentials and sender details in Render's environment settings to enable real email delivery.
- After the first successful deployment, create the initial production Owner with `python manage.py createsuperuser` in the service shell.
- Configure a custom domain and set SITE_URL to its HTTPS URL if required; keep all credentials out of source control.

8. Known limitations and operational notes

- The accounting summary is not a full profit statement; it does not deduct fuel purchase costs, taxes, or other liabilities.
- Real email delivery depends on an SMTP provider and its correct configuration.
- The Render resources described by the Blueprint are not created until the owner applies it in Render; plans may incur charges.
- Production backups, access management, monitoring, and recovery procedures should be configured before real business use.

Counts are measured from the current codebase; test totals refer to the latest recorded full-suite run. This summary describes the project implementation and deployment preparation. It is not a financial, tax, or production-readiness certification.